

PQC SECURE FILE TRANSFER

**Secure Client-Server File Transfer using ML-KEM-768, HKDF-SHA256,
AES-256-GCM and SHA-256**

Project Report

Prepared by: Chetna Mundra | Jyotika Manda | Swastik Garg

Github Repository Link:

<https://github.com/Chetna-mundra/PQC-Secure-File-Transfer.git>

Table of Contents

1. Introduction	2
2. Technical Background	2
3. System Requirements and Final Configuration	4
4. System Architecture	5
5. Custom ML-KEM Implementation	6
6. TCP Message Framing and Network Protocol	7
7. File Encryption and Integrity Workflow	9
8. Graphical User Interface	10
9. Integration and Demonstration Module	13
10. Validation, Testing Evidence and Defensive Checks	13
11. Security Analysis	15
12. Limitations and Future Improvements	17

13. Conclusion

17

1. Introduction

1.1 Project Objective

Modern secure communication commonly uses public-key mechanisms for key establishment and symmetric cryptography for bulk data protection. Large-scale quantum computers would threaten many currently deployed public-key schemes, motivating the development and migration toward post-quantum cryptography (PQC). NIST standardized ML-KEM in FIPS 203 in August 2024 as a post-quantum key-encapsulation mechanism whose security is related to the Module Learning With Errors problem.

This project applies this idea to a concrete systems problem: securely transferring a file between a client and a server while demonstrating how a post-quantum KEM can establish a key without making it public.

The main objectives of the project are:

- Implement the core ML-KEM algorithm using Python, rather than using the library.
- Use ML-KEM-768 in an actual TCP client-server workflow to establish a shared secret.
- Derive an AES-256 key from the shared secret using HKDF-SHA256.
- Protect file contents with AES-256-GCM authenticated encryption and independently verify the file SHA-256 digest.
- Implement explicit message framing over TCP and defensive checks for sizes and filenames.
- Provide both command-line operation and a Tkinter client GUI.

2. Technical Background

2.1 Key encapsulation mechanisms

A key encapsulation mechanism (KEM) allows communicating parties to establish a shared secret over a public channel. In ML-KEM, the main public operations are KeyGen, Encaps and Decaps. The shared secret is not itself a file-encryption algorithm; it generates the keys which are then used by symmetric cryptography algorithms for file encryption.

$$(ek, dk) = ML\text{-}KEM.KeyGen()$$

$$(K, c) = ML\text{-}KEM.Encaps(ek)$$

$$K' = ML\text{-}KEM.Decaps(dk, c), \text{ with } K' = K \text{ for a valid exchange}$$

2.2 Module-LWE and polynomial ring setting

ML-KEM is a module-lattice construction. This project uses the standardized polynomial degree $n = 256$ and modulus $q = 3329$. Polynomial arithmetic is performed in the quotient ring shown below. Vectors and matrices of such ring elements provide the module structure used by the underlying public-key encryption component.

$$R_q = \mathbb{Z}_q[X] / (X^{256} + 1), \quad q = 3329$$

The Learning With Errors problem hides linear algebraic structure by adding small errors. An LWE-style relation may be written as: $b = A s + e \pmod{q}$

Module-LWE replaces scalar/vector entries with polynomial-ring elements.

2.3 Number Theoretic Transform

Polynomial multiplication is central to ML-KEM. Naive polynomial multiplication takes quadratic time complexity which is why the Number Theoretic Transform (NTT) changes representation so that polynomial products can be carried out efficiently using structured base-case operations. We implement forward NTT, inverse NTT, and NTT-domain multiplication as part of the K-PKE layer.

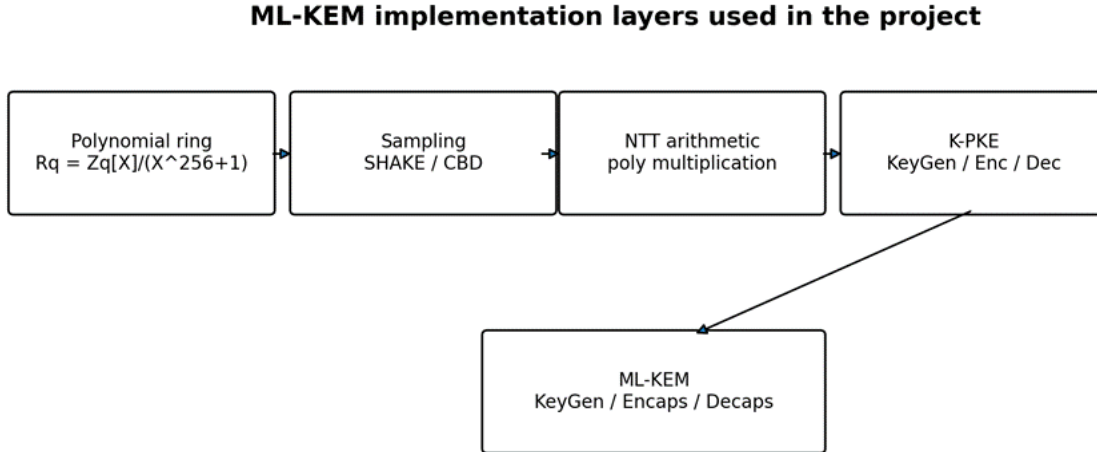


Figure 1. Layered view of the custom ML-KEM implementation.

2.4 HKDF-SHA256

HKDF is an HMAC-based key derivation function based on an extract-then-expand design. In the final network implementation, the 32-byte ML-KEM shared secret is input to HKDF instantiated with SHA-256. The requested output length is 32 bytes, hence producing a clean application-specific AES key.

$$K_{AES} = \text{HKDF-SHA256}(K, \text{salt} = \text{None}, \text{info} = \text{"PQC-Secure-File-Transfer"}, \text{Length} = 32)$$

2.5 AES-256-GCM and associated data

Galois/Counter Mode (GCM) is an authenticated-encryption mode that provides confidentiality and authentication, and it can authenticate associated data that remains visible in plaintext. The project uses a 32-byte AES key and a 12-byte random nonce. Metadata bytes are passed as associated authenticated data (AAD), binding the visible filename/size/nonce metadata to the ciphertext authentication check.

2.6 SHA-256

SHA-256 produces a 32-byte message digest. The project computes the digest before encryption, encrypts digest || file bytes, and has the server recompute the digest after decryption. AES-GCM already provides ciphertext authentication; the SHA-256 value is therefore used as an explicit content-integrity digest and as a value returned to the client for end-to-end comparisons.

3. System Requirements and Final Configuration

3.1 Repository layout

Path	Role
mlkem.py	Custom ML-KEM implementation including internal K-PKE
network/protocol.py	Four-byte length-prefixed TCP framing helpers: send_message(), recv_message() and exact reads with recv_exact(), with per-call maximum message sizes.
network/client.py	Key generation, KEM exchange, HKDF key derivation, AES-GCM encryption, file transfer and response verification.
network/server.py	Encapsulation, HKDF key derivation, metadata and filename validation, decryption, digest check, safe file storage and encrypted response.
app_gui.py	Tkinter graphical client built on network.client.send_file().
integration.py	Single-process local ML-KEM + AES-GCM demonstration; not the TCP protocol.
received_files/	Server-side output directory created/used for successfully verified files.
README.md	Setup, running instructions and implementation notes.

Table 2. Major project files and responsibilities.

Responsibility for the repository follows these layers. mlkem.py is the custom ML-KEM implementation (Section 5). The networking layer (network/protocol.py, network/client.py and network/server.py), implemented by Swastik Garg, turns the cryptographic building blocks into a working client-server file-transfer system: it adds length-prefixed message framing, integrates the ML-KEM key exchange, derives the session key with HKDF-SHA256, protects the file with AES-256-GCM and SHA-256 verification, and validates and bounds everything it receives (Sections 6, 7 and 10). app_gui.py builds on network.client.send_file(), and integration.py is a standalone demonstration of the ML-KEM and AES-GCM steps (Sections 8 and 9).

4. System Architecture

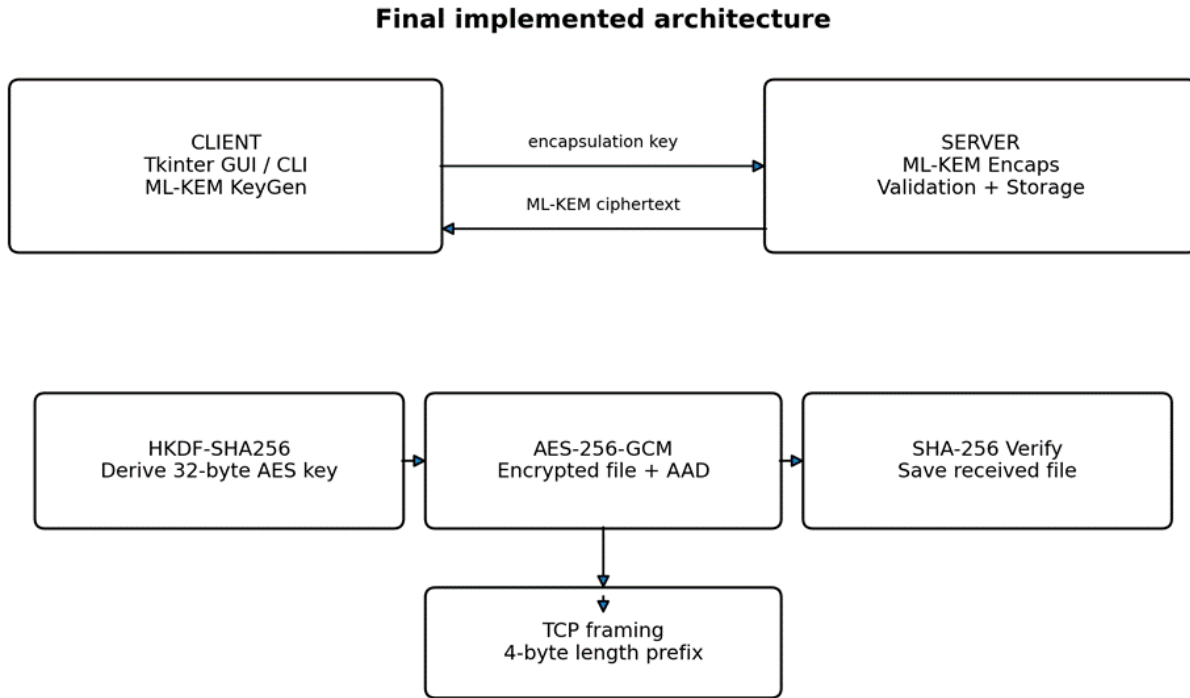


Figure 2. Final system architecture.

The architecture deliberately separates three concerns: (1) post-quantum key establishment, (2) symmetric authenticated file protection, and (3) reliable message transfer over TCP. The networking layer (`network/protocol.py`, `network/client.py`, `network/server.py`) connects them: it calls the `KeyGen/Encaps/Decaps` interface of `mlkem.py`, derives the session key with HKDF-SHA256, protects the file with AES-256-GCM and moves every message over TCP using the framing described in Section 6. HKDF and AES-GCM come from the Python cryptography package, and SHA-256 and the digest comparison use the standard-library `hashlib` and `hmac` modules; only ML-KEM is implemented from scratch. Neither endpoint authenticates the other (Section 11.2). The GUI is only a front end to the same client transfer function used by the terminal workflow.

4.1 Client responsibilities

- Validate that the selected path exists, is a regular file, and is within the 100 MiB limit.
- Generate a fresh ML-KEM-768 key pair for each call to `send_file()`.
- Send the encapsulation key (1184 bytes) and receive the ML-KEM ciphertext, accepting only the exact expected size (1088 bytes).
- Decapsulate to recover the 32-byte shared secret and derive the AES key with HKDF-SHA256.

- Read the file, compute SHA-256, generate a fresh random 12-byte nonce, build the metadata (filename, encrypted size, nonce) and AES-GCM encrypt digest || file with the metadata bytes as AAD.
- Send metadata and encrypted payload, then authenticate/decrypt the server response, require a SUCCESS status and verify the returned digest. Any failure is raised as an exception, and the socket is always closed.

4.2 Server responsibilities

- Listen for TCP connections (127.0.0.1:5000 by default) and apply a 30-second timeout to each accepted connection.
- Receive the ML-KEM encapsulation key (exactly 1184 bytes), encapsulate a shared secret with the `mlkem.py` Encaps function, which applies its own checks to the key, and return the 1088-byte ciphertext.
- Derive the same AES key via HKDF-SHA256.
- Validate the metadata (exact key set, filename, size and nonce), receive the declared encrypted payload, and AES-GCM authenticate/decrypt it using the metadata bytes as AAD.
- Recompute SHA-256 and compare it to the decrypted expected digest using `hmac.compare_digest()`.
- Save only verified file data; avoid silent overwrite by generating a numbered filename when necessary.
- Send an AES-GCM-protected JSON result back to the client; failures after key derivation are reported the same way as short ERROR messages, while detailed diagnostics stay on the server console.

5. Custom ML-KEM Implementation

The ML-KEM module contains the implementation aligned to the algorithm organization in FIPS 203. It uses Python standard-library hashing/XOF support and operating-system randomness.

5.1 Supported parameter sets

Parameter set	k	eta1	eta2	du	dv	Encapsulation key	Decapsulation key	Ciphertext	Shared secret
ML-KEM-512	2	3	2	10	4	800 B	1632 B	768 B	32 B
ML-KEM-768	3	2	2	10	4	1184 B	2400 B	1088 B	32 B
ML-KEM-1024	4	2	2	11	5	1568 B	3168 B	1568 B	32 B

Table 3. ML-KEM parameter sets and serialized sizes [1].

5.2 Auxiliary algorithms

The implementation includes bit/byte conversion, polynomial encoding and decoding, rejection sampling for NTT coefficients, centered-binomial sampling, NTT, inverse NTT, base-case multiplication, and

SHA3/SHAKE-based helper functions. These routines support K-PKE, which is used internally by ML-KEM rather than exposed as the application encryption mechanism.

5.3 Public API

```
def keygen(self):
    return self.keygen_internal(os.urandom(32), os.urandom(32))

def encaps(self, ek: bytes):
    ek = _require_bytes(ek, self.ek_size, "encapsulation key")
    ...
    return self.encaps_internal(ek, os.urandom(32))

def decaps(self, dk: bytes, c: bytes) -> bytes:
    ...
    return self.decaps_internal(dk, c)
```

The final public methods add size and consistency checks around deterministic internal routines. Applications use `keygen()`, `encaps()`, and `decaps()`; internal deterministic inputs exist to make the algorithm structure inspectable and testable.

5.4 Why ML-KEM-768 was selected for networking

Although the class supports all three parameter sets, the networking code fixes `PARAMETER_SET = "ML-KEM-768"`. This provides the middle standardized parameter set and keeps the demonstration consistent across client, server, GUI labels and expected serialized sizes.

6. TCP Message Framing and Network Protocol

6.1 Why framing is necessary

TCP provides an ordered byte stream rather than application-level messages. A receiver cannot assume that one call to `recv()` returns exactly one sender message. The project therefore sends each logical message as a four-byte unsigned big-endian length followed by exactly that many message bytes. This is called length-prefix framing.

wire message = uint32_be(length) // payload

```
header = struct.pack("!I", len(data))
sock.sendall(header + data)

header = recv_exact(sock, 4)
length = struct.unpack("!I", header)[0]
return recv_exact(sock, length)
```

`send_message()` prepends the header to the data and transmits it with `sendall()`; it rejects data that is not bytes, or that exceeds the caller's maximum, before sending anything. `recv_exact(sock, n)` loops on `recv()` until exactly `n` bytes have been collected, because a single `recv()` may return fewer bytes than requested. If the peer closes the connection first, it raises `ConnectionError` instead of returning a truncated message.

The receiver checks the announced length against a caller-supplied maximum before reading the body. This prevents the protocol from blindly accepting an arbitrarily large claimed message size. The four-byte header can express up to 4 GiB, so `protocol.py` defines a default ceiling of 100 MiB (`MAX_MESSAGE_SIZE`) and each call site passes a tighter limit for the message it expects (Table 4).

6.2 Final handshake and transfer sequence

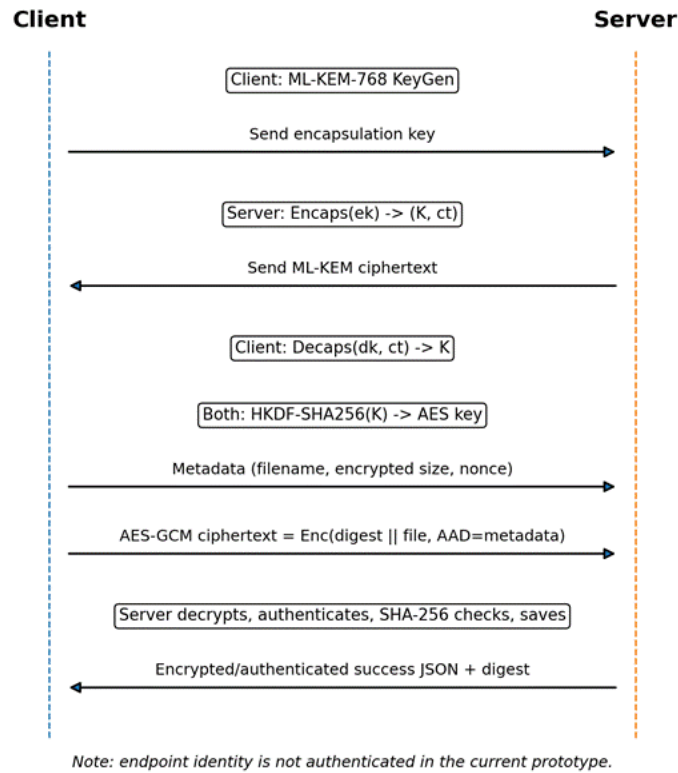


Figure 3. Final client-server sequence implemented by the current networking code.

6.3 Message order

1. Client sends the ML-KEM-768 encapsulation key (1184 bytes).
2. Server runs $\text{Encaps}(ek)$, keeps the 32-byte shared secret, and sends the 1088-byte ML-KEM ciphertext.
3. Client runs $\text{Decaps}(dk, \text{ciphertext})$ to recover the shared secret.
4. Both sides independently derive the same 32-byte AES key using HKDF-SHA256.
5. Client sends JSON metadata (filename, encrypted size, nonce) as a framed plaintext message.
6. Client sends one AES-GCM encrypted payload (SHA-256 digest || file) as a second framed message.
7. Server verifies/decrypts, stores the file, and returns an encrypted/authenticated JSON result.
8. Client authenticates/decrypts the response, requires status SUCCESS, and compares the returned SHA-256 value with the digest of the file it sent.

None of these steps authenticates the identity of either endpoint. The authenticated response in step 7 shows only that the peer holds the same session key; it does not prove who that peer is (Section 11.2).

6.4 Message size limits

Every receive call passes a maximum size, so a peer cannot make the receiver read an unbounded message. Table 4 lists the limits used by the final code (ML-KEM-768 sizes).

Message	Direction	Limit enforced before the body is read
---------	-----------	--

ML-KEM encapsulation key	Client to Server	Exactly 1184 bytes; the receive limit equals the expected key size.
ML-KEM ciphertext	Server to Client	Exactly 1088 bytes; the receive limit equals the expected ciphertext size.
Metadata (JSON)	Client to Server	At most 4 KiB.
Encrypted payload	Client to Server	At most 32 + 100 MiB + 16 bytes, and exactly the size declared in the metadata.
Server response (JSON envelope)	Server to Client	At most 64 KiB.

Table 4. Message size limits enforced by the framing layer.

Because each limit is checked against the length header before `recv_exact()` is called, an oversized or falsified length is rejected without the receiver reading or allocating that many bytes. These checks protect the receiver from malformed input; they do not authenticate the sender.

7. File Encryption and Integrity Workflow

7.1 Metadata

The final plaintext metadata contains exactly the filename, encrypted payload size, and the AES-GCM nonce encoded as hexadecimal. The SHA-256 digest is not included in plaintext metadata in the current version. The size field is the encrypted payload size (file size + 32 + 16), not the original file size. The server rejects metadata with missing or additional keys. Because the metadata bytes are the AES-GCM AAD, they remain visible but cannot be altered without making authentication fail.

```
metadata = {
    "filename": file_path.name,
    "size": encrypted_size,
    "nonce": nonce.hex()
}
metadata_bytes = json.dumps(metadata, separators=(",", ":")).encode("utf-8")
```

7.2 Encrypted payload

$$P = \text{SHA256}(\text{file}) \parallel \text{file_bytes}$$

$$C = \text{AES-256-GCM}.\text{Encrypt}(K_{\text{AES}}, \text{nonce}, P, \text{AAD} = \text{metadata_bytes})$$

For a file of F bytes, the encrypted payload size is $F + 32 + 16$ bytes: 32 bytes for the embedded SHA-256 digest and 16 bytes for the GCM authentication tag. The nonce is sent in metadata rather than prefixed to the ciphertext. It is generated with `os.urandom(12)` for each transfer, and a new AES key is derived for every connection. HKDF and AES-GCM are the implementations in the Python cryptography package; the networking layer defines how they are used: the HKDF info string, the payload layout (digest || file), the AAD and the order of the checks.

7.3 Server-side verification

The server treats everything received from the client as untrusted and checks it in this order:

- The metadata is valid UTF-8 JSON and a dictionary with exactly the keys filename, size and nonce.

- The filename passes `sanitize_filename()` (Table 6).
- The size is an integer (not a boolean) between 48 bytes (digest plus tag) and 32 bytes + 100 MiB + 16 bytes, and the nonce is a 24-character hexadecimal string that decodes to 12 bytes.
- The encrypted payload is received under the same upper limit, and its length must equal the declared size.
- AES-GCM must authenticate the payload with the metadata bytes as AAD; the decrypted payload must contain at least the 32-byte digest and no more than the maximum file size.

```
decrypted_payload = AESGCM(aes_key).decrypt(
    nonce, encrypted_data, metadata_bytes
)
expected_digest = decrypted_payload[:32]
decrypted_data = decrypted_payload[32:]
actual_digest = hashlib.sha256(decrypted_data).digest()
if not hmac.compare_digest(actual_digest, expected_digest):
    raise ValueError("SHA-256 verification failed")
```

The file is written only after GCM authentication and the explicit SHA-256 comparison succeed. Files are created in exclusive-create mode, so an existing file is never silently overwritten: if “name.ext” already exists, the server tries “name (1).ext”, “name (2).ext” and so on up through counter 100, and reports an error if no free name is found. The name actually used is returned to the client in the response.

7.4 Authenticated server response

The server serializes a result dictionary, encrypts it under AES-GCM with a fresh random 12-byte nonce and fixed response-specific AAD, and sends an envelope containing nonce and ciphertext. The client decrypts/authenticates the response, requires status SUCCESS and refuses success if the returned SHA-256 value differs from its original file hash.

Failures after the session key exists (invalid metadata, an AES-GCM authentication failure, a digest mismatch) are reported through the same encrypted envelope as an ERROR result with a short message; an authentication failure always produces the fixed text “Decryption/authentication failed”, and unexpected exception types are replaced by a generic message while the details stay on the server console. If a failure occurs before the key exists (for example an invalid encapsulation key), no response is sent and the server closes the connection, so the client reports a lost connection. The encrypted response shows that the sender holds the session key; it does not identify the server (Section 11.2).

8. Graphical User Interface

The client graphical interface is implemented in `app_gui.py` using Python's standard *tkinter* library. It provides an interactive front end on top of `network.client.send_file()`, allowing users to select files, configure the connection, and transfer data securely without typing commands into the terminal.

8.1 Visual Pipeline and UI Design

A key requirement for the client interface was to provide a clear visual demonstration of the post-quantum transfer process while it is running. To achieve this, the GUI includes:

- *Background Threading*: Tkinter's interface runs on a single event loop (`window.mainloop()`). If network communication and cryptographic calculations run directly on this thread, the window freezes. To prevent this, the transfer function is run inside a background daemon thread (`threading.Thread(target=run_network, daemon=True)`). This keeps the window fully responsive so progress badges and log text can update in real time.

- **4-Stage Progress Badges:** Four badges sit at the top of the window to show exactly where the protocol is:

1. Connect (opening the socket connection)
2. ML-KEM (sending the 1184-byte public key and decapsulating the 1088-byte ciphertext)
3. AES-GCM (deriving the AES key with HKDF and encrypting the payload)
4. Check Hash (verifying that the server's returned SHA-256 digest matches)

These badges start inactive gray, turn bright blue while the step is running and switch to green once completed.

- **Live Artifacts Box:** A dedicated status card displays runtime cryptographic parameters: the ML-KEM key exchange status (1088-byte ciphertext), the AES-256-GCM construction parameters (12-byte nonce/IV and 16-byte tag), and the first 26 characters of the verified SHA-256 digest.
- **Fixed Window Size:** The window size is locked to 720x640 using `window.resizable(False, False)` so that expanding or full-screening the window does not stretch elements across high-resolution displays.

GUI element	Behavior
Server Address (Host / Port)	Defaults to 127.0.0.1 and 5000 for testing and ensures the port entered is an integer before attempting to connect; user-editable.
File Selector (Browse File)	Uses a standard file-selection dialog for clicking and selecting any file instead of typing out long file paths manually.
Send Button (Encrypt & Send File)	Starts the transfer on a separate background thread that calls <code>send_file()</code> so the screen does not freeze, locking the button while sending to avoid accidental duplicate clicks and automatically re-enables in a <i>finally</i> block once finished
Live Artifacts Box	Displays real-time cryptographic details as they happen, showing the 1088-byte KEM ciphertext status, the AES-GCM IV and tag sizes, and the verified SHA-256 hash.
Activity Log (ScrolledText)	A terminal-style output area that scrolls down automatically as each step finishes, printing connection details, packet sizes, and server responses.
Status Label and Alerts	Shows the overall transfer state (Ready, Sending..., Done!) and pops up an alert box confirming success or explaining errors

Table 5. GUI controls and status presentation.

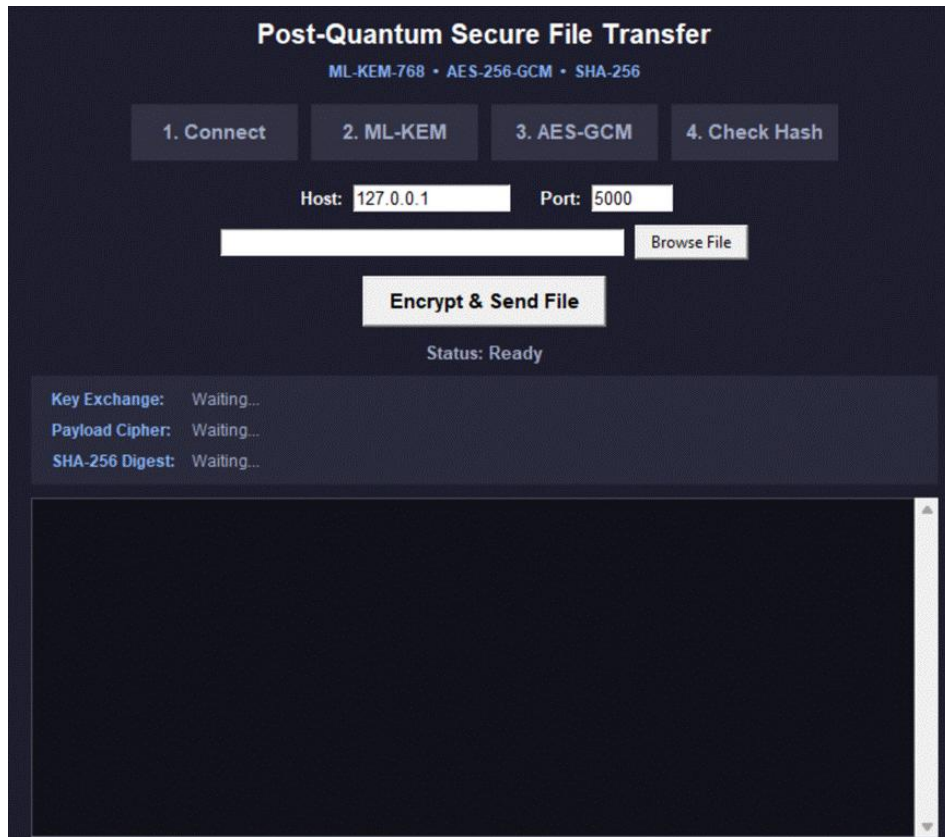


Figure 4. GUI before transfer

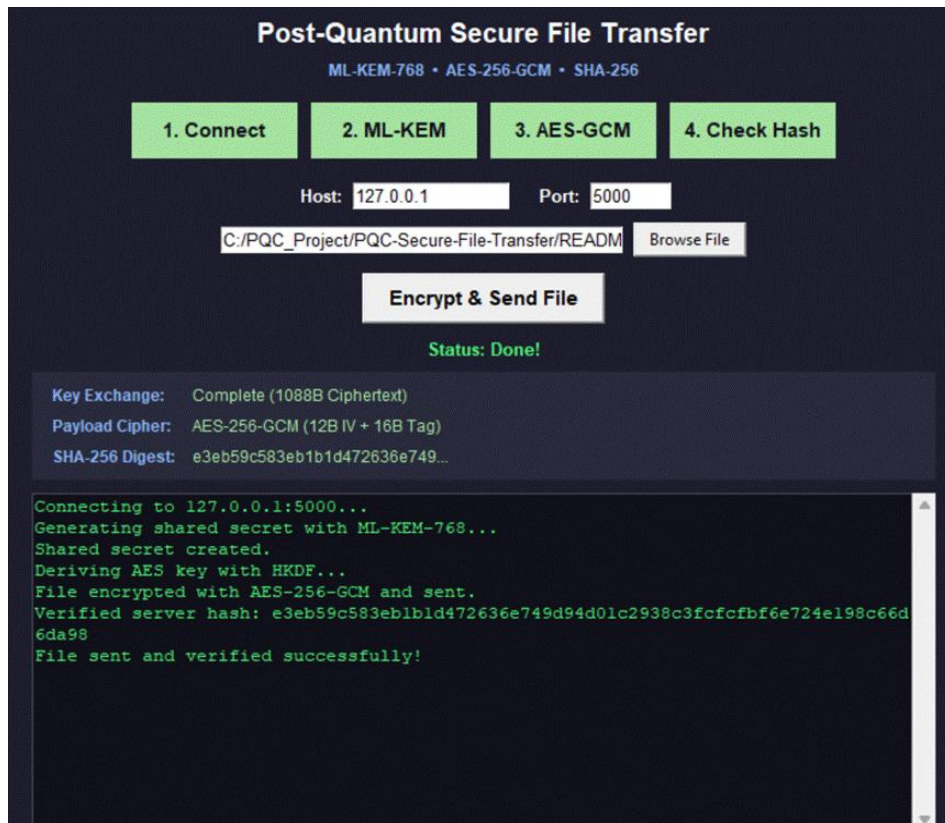


Figure 5. GUI after a successful transfer

9. Integration and Demonstration Module

The integration.py script was written as a local, single-process testing tool for the cryptographic components. It runs the entire cryptographic pipeline in memory to confirm that our ML-KEM implementation, AES-256-GCM encryption, and SHA-256 hashing work properly together before dealing with network sockets.

9.1 Verification Workflow

The script runs through the cryptographic steps sequentially:

- **Key Generation:** Creates an ML-KEM-768 key pair, generating a public encapsulation key client_ek (1184 bytes) and a private decapsulation key client_dk (2400 bytes).
- **Encapsulation & Decapsulation:** Simulates a key agreement exchange. The server part runs kem.encaps(client_ek) to produce a 32-byte shared key and a 1088-byte ciphertext; the client part runs kem.decaps(client_dk, ciphertext) to recover the shared key.
- **Key Agreement Check:** Verifies that server_shared_key == client_shared key, confirming that both sides arrived at the exact same secret.
- **AES-256-GCM Encryption:** Takes user input (or fallback sample text), encrypts it with AESGCM using the shared key and a random 12-byte nonce (os.urandom(12)), and then decrypts it to ensure the message recovers cleanly.
- **SHA-256 Digest Check:** Computes hashlib.sha256() on both the original input and the decrypted message to prove that data integrity is maintained throughout.

```
server_shared_key, ciphertext = kem.encaps(client_ek)
client_shared_key = kem.decaps(client_dk, ciphertext)

if server_shared_key == client_shared_key:
    print("key agreement verified as the shared secret matches.")
```

9.2 Difference from the Network Application

There is one deliberate difference between integration.py and the final client-server code:

- In integration.py, the raw 32-byte ML-KEM shared secret is used directly as the AES-256 key to test the components simply and quickly in memory.
- In the full network application (client.py and server.py), the shared secret is first passed through HKDF-SHA256 to derive the session AES key.

Writing this standalone test script made debugging much easier by separating pure cryptographic issues from network-level bugs like byte framing, socket timeouts, and connection errors.

10. Validation, Testing Evidence and Defensive Checks

10.1 ML-KEM smoke test reproduced from the current source

The current mlkem.py contains a built-in smoke test that generates a key pair, encapsulates, decapsulates, and raises an error if the two shared keys differ. While preparing this report, that exact module was executed and produced the following output:

```
ML-KEM-512: shared keys match; ek=800, dk=1632, ciphertext=768 bytes
ML-KEM-768: shared keys match; ek=1184, dk=2400, ciphertext=1088 bytes
ML-KEM-1024: shared keys match; ek=1568, dk=3168, ciphertext=1568 bytes
```

This evidence verifies the built-in key-agreement path and serialized sizes for the current module.

10.2 Implemented defensive checks

Area	Implemented check / behavior
Client file input	Existence, regular-file status and the 100 MiB maximum size, checked before connecting and again after reading the file.
KEM messages	Receive limits equal to the expected sizes (1184-byte key, 1088-byte ciphertext) followed by an exact length check on both sides; mlkem.py applies its own input checks.
Protocol framing	Maximum message length enforced before the body is read (Table 4); non-bytes or oversized sends are rejected; a connection closed mid-message raises ConnectionError.
Metadata	Valid UTF-8 JSON object with exactly the keys filename, size and nonce; size an integer from 48 bytes to 32 + 100 MiB + 16 bytes; nonce a 24-character hex string (12 bytes); received payload length must equal the declared size.
Filename	Rejects non-string or empty names, “.” and “..”, names over 255 characters, control characters, “/”, “\” and “:”, a trailing space or period, and Windows reserved device names (CON, PRN, AUX, NUL, COM1-COM9, LPT1-LPT9, with or without an extension). Separators are rejected rather than normalised, so a name cannot leave received_files/.
AES-GCM	An invalid authentication tag (modified ciphertext, metadata or key) fails the transfer with “Decryption/authentication failed”; nothing is written.
Digest	Server recomputes SHA-256 and compares using hmac.compare_digest(); decrypted payloads shorter than 32 bytes are rejected.
Output	Exclusive create mode (“xb”) prevents silent overwrite; up to 100 numbered alternatives are tried; files are written only after authentication and the digest check.
Server response	Encrypted/authenticated (AES-GCM with a response-specific AAD) before the client accepts it; the client also requires status SUCCESS and a matching SHA-256.
Error reporting	Client functions raise exceptions on failure; the server returns short error messages and keeps detailed diagnostics on its console.
Sockets	30-second timeout on the client socket and on each accepted server connection (per socket operation; accept() and total transfer time are not bounded). The listening socket uses SO_EXCLUSIVEADDRUSE where available and SO_REUSEADDR otherwise.

Table 6. Defensive checks visible in the final source code.

These checks are documented from the source code. The supplied source tree contains no automated test suite for the networking layer, so the evidence of correct operation is the end-to-end run in Section 10.3.

10.3 End-to-end evidence

Figures 5 to 7 come from one end-to-end transfer of README.md on 127.0.0.1:5000. In Figure 6 the server receives the 1184-byte key, returns the 1088-byte ciphertext, derives the AES key, receives metadata declaring an encrypted size of 15100 bytes, receives 15100 bytes, passes AES-GCM authentication, verifies SHA-256 and saves the 15052-byte file ($15100 = 15052 + 32 + 16$). The SHA-256 value printed by the server is

the same value the client GUI reports as the verified server hash in Figure 5, and Figure 7 shows the saved file in received_files. This run demonstrates the successful path only.

```
C:\PQC_Project>"C:\Program Files\Python313\python.exe" c:/PQC_Project/PQC-Secure-File-Transfer/network/server.py
=====
PQC SECURE FILE TRANSFER SERVER
=====

[+] ML-KEM parameter set: ML-KEM-768
[+] Listening on 127.0.0.1:5000

[+] Waiting for client...
[+] Client connected: ('127.0.0.1', 50167)

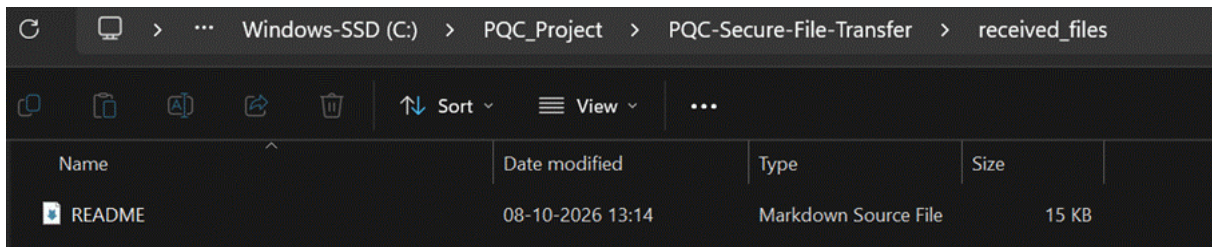
[1/6] Receiving ML-KEM public key...
      Received 1184 bytes
[2/6] Performing ML-KEM encapsulation...
      ML-KEM ciphertext: 1088 bytes
      Shared secret: 32 bytes
      Ciphertext sent to client.
[3/6] Deriving AES-256 key with HKDF...
      AES key derived.
[4/6] Receiving file metadata...
      Filename: README.md
      Encrypted size: 15100 bytes
[5/6] Receiving encrypted file...
      Received: 15100 bytes
      Decrypting using AES-256-GCM...
      AES-GCM authentication successful.
[6/6] Verifying SHA-256...

=====
TRANSFER SUCCESSFUL
=====
File saved to: C:\PQC_Project\PQC-Secure-File-Transfer\received_files\README.md
File size: 15052 bytes
SHA-256: e3eb59c583eb1b1d472636e749d94d01c2938c3fcfcfbf6e724e198c66d6da98
=====

[+] Client connection closed.

[+] Server stopped.
```

Figure 6. Server terminal during successful transfer



Name	Date modified	Type	Size
README	08-10-2026 13:14	Markdown Source File	15 KB

Figure 7. Received file verification

11. Security Analysis

11.1 What the project protects

- **ML-KEM** helps the client and server establish a shared secret using a method designed to resist known quantum attacks [1].
- **AES-256-GCM** encrypts the file and detects changes to the encrypted data and its attached metadata [2].

- **SHA-256** creates a file fingerprint, which the server checks after decryption [4].
- The server's confirmation message is also encrypted and checked for tampering. This shows that the responder holds the session key; it does not identify the server (Section 11.2).
- Message-size limits, metadata and nonce validation, filename checks and exclusive file creation help reject malformed or oversized input and prevent accidental overwriting; the length checks run before message bodies are read.
- A fresh key pair and encapsulation are used for every connection and the AES key is derived from that session's shared secret, so a recorded metadata/ciphertext pair is not expected to authenticate under a later session's key.

11.2 The connection does not verify identity

The current system establishes a shared secret, but it does not verify who the client or server actually is. An attacker positioned between them could replace the key-exchange messages and establish a separate secret with each side. The attacker could then read and re-encrypt the transferred data. This is called a man-in-the-middle attack.

ML-KEM alone does not authenticate either endpoint: it only lets two parties agree on a secret. Here the encapsulation key is sent unauthenticated, so the server cannot tell who sent it and the client cannot tell who answered. The encrypted server response and the SHA-256 comparison show that the peer holds the session key and that the file arrived unmodified from that peer, not that the peer is the intended server. The server also performs no client authentication or authorization.

To prevent this, a future version needs a trusted way to verify identity, such as digital signatures or certificates. The current prototype does not provide this protection.

11.3 The implementation is for learning

We wrote ML-KEM in Python to understand how it works. It has not undergone the security testing or validation needed for real-world deployment. The code is also not constant-time, meaning its execution time may depend on secret values and could reveal information. Python does not guarantee that secret keys are completely erased from memory after use.

11.4 Nonces and encryption keys

AES-GCM uses a nonce, a value that must never repeat with the same encryption key. Our project generates a random 12-byte nonce for the file and another for the server's response. Repetition is very unlikely in this small demonstration, but random generation does not guarantee uniqueness.

Both directions currently use the same AES key. A stronger design could use separate keys for each direction and a method that guarantees nonces do not repeat.

11.5 Some file details remain visible

The filename, encrypted file size and nonce are sent without encryption, so someone observing the connection can see them. These details are included in the AES-GCM authentication check. Changing them should cause verification to fail, but their contents are not hidden.

11.6 Scope of the defensive checks

The checks in Table 6 reject malformed, oversized and unsafe input, but they are deliberately limited. The 30-second timeout applies to each socket operation, so it does not bound the total time a slow peer can occupy the single-threaded server, and `accept()` has no timeout. The server handles one client at a time. The whole file is held in memory, up to the 100 MiB limit. The filename check rejects specific unsafe patterns rather than enforcing every operating-system naming rule, so a name with characters that a file system disallows can still make the file write fail.

12. Limitations and Future Improvements

- *Identity verification*: The client and server do not verify each other's identity. Adding digital signatures, certificates or trusted keys would help prevent man-in-the-middle attacks.
- *Separate encryption keys*: The file and server response currently use the same AES key. Separate keys could be used for each direction.
- *Key derivation*: Future versions could include a hash of the initial key-exchange messages when deriving keys, alongside identity verification.
- *Large-file handling*: The entire file (up to the 100 MiB limit) is currently loaded into memory. Processing it in smaller encrypted chunks would support larger files more efficiently.
- *Multiple clients*: The server handles one client at a time and, by default, returns after the first connection, including a failed one; with `once=False` it keeps serving clients one after another. It could be extended to handle multiple clients concurrently.
- *Connection limits*: Timeouts apply per socket operation rather than to the whole transfer, and clients are neither authenticated nor rate-limited. A total per-connection deadline and basic rate limiting would harden the server.
- *Implementation security*: Our Python ML-KEM implementation is intended for learning. Real-world use would require a thoroughly reviewed cryptographic implementation.
- *Protocol updates*: Adding version information to messages would make future changes easier to manage.
- *More testing*: Automated tests could check individual functions, standard examples with known results, complete file transfers and how the system handles errors.

13. Conclusion

The project demonstrates a working software architecture for protected file transfer. The custom ML-KEM implementation exposes standardized KeyGen, Encaps and Decaps operations and supports all three ML-KEM parameter sets; the integrated network application uses ML-KEM-768. TCP framing makes the byte stream usable as discrete application messages, HKDF-SHA256 converts the KEM output into a 256-bit AES key, AES-256-GCM protects the file and authenticates metadata, and SHA-256 supplies an explicit content digest that is checked by both sides.